Companion Proofs:
If It Walks Like an Arbitrage: Protocol-Agnostic Detection with Decidable Structural Equivalence

Anonymous Authors

This document collects the full proofs and the practical structural-equivalence material that accompany the submission. Section 1 contains the mechanized proofs of Theorems 1–5; Section 2 develops the structural-equivalence query language with worked examples. Cross-references prefixed with `sec:` or `def:` that resolve outside this document refer to the main paper.

1. Full Proofs

We prove the five theorems stated in Section 3.6. Throughout, we use the notation of Definitions 1–5 and the algorithms of Figures 4–7. We write $|T|$ for the number of nodes in an AST T, and $children(T)$ for the multiset of children of the root of T.

1.1. Proof of Theorem 1 (Preservation)

We first establish a loop invariant that underpins both this theorem and the soundness proof.

Definition 1 (Walk correspondence)**.** *A chain node C in an AST T* corresponds *to a walk $W = e_1, e_2, \ldots, e_m$ in $G = (V, E)$ if each $e_i \in E$ and the address sequence $s(e_1), d(e_1){=}s(e_2), \ldots, d(e_m)$ is continuous. For a merged node representing parallel paths $C_1, \ldots, C_p$, the node corresponds to the set $\{W_1, \ldots, W_p\}$ where each W_i is a walk for C_i.*

Proof. We proceed by induction on the number of rewrite rule applications, starting from the initial AST T_0 produced by BUILD-AST.

Base case (T_0). Every Leaf node $\ell = \mathsf{Leaf}(s, d, a, \tau, \sigma)$ in T_0 is constructed by the decode layer from an event in the execution trace (Definition 2). Each such event corresponds to exactly one edge $(s, d, a, \tau, \sigma) \in E$ in the transfer graph (Definition 1). A single edge is a walk of length 1, so the invariant holds for T_0.

Inductive step. Assume that every leaf and every chain in the current AST satisfies the walk correspondence (leaves trivially, by the base case). We show that each rewrite rule preserves it.

1) **Chaining (Algorithm 4, Rule 1).** Let ℓ_1, ℓ_2 be siblings with $d(\ell_1) = s(\ell_2)$. By hypothesis, ℓ_1 corresponds to a walk $W_1 = e_1, \ldots, e_j$ ending at $d(e_j) = d(\ell_1)$, and ℓ_2 to a walk $W_2 = e_{j+1}, \ldots, e_k$ starting at $s(e_{j+1}) = s(\ell_2)$. Since $d(\ell_1) = s(\ell_2)$, we have $d(e_j) = s(e_{j+1})$, so the concatenation $W_1 \cdot W_2 = e_1, \ldots, e_k$ is a continuous walk in G. The chain node records this concatenation. No edge outside E is introduced: every e_i belongs to W_1 or W_2, both subsets of E.
   The *pool check* ($\sigma \neq d(\ell_1)$) and the *token-mismatch check* ($\tau(\ell_1) \neq \tau(\ell_2)$) are preconditions that restrict *when* chaining fires, but do not alter the walk: when satisfied, the resulting chain still records the same edges. The same argument applies to the transitive extension (chain + leaf, chain + chain): each operand corresponds to a walk by hypothesis, and the concatenation is valid whenever the junction addresses match.
2) **Lifting (Algorithm 4, post-loop).** A fully reduced Tree node $n = \mathsf{Tree}(c, [t_1, \ldots, t_m])$ is dissolved: its children $t_1, \ldots, t_m$ become children of n's parent. This operation changes only the tree structure; no chain's internal transfer sequence is modified. Formally, for every chain C that was a descendant of n, the walk W associated with C before lifting is identical to the walk after lifting. Therefore the invariant is preserved.
3) **Merging (Algorithm 5, R_7/R_8/R_9/R_{13}).** Two chains C_1, C_2 at the same tree level are merged into a single node C_m whenever they share endpoints ($s(C_1) = s(C_2)$, $d(C_1) = d(C_2)$) and satisfy one of the four merge premises: R_7 (parallel-path, distinct τ_{in}), R_8 (closed cycles with matching tokens *or* $BalCont$ at the origin), R_9 (closed cycles sharing only τ_{in}), or R_{13} (full-token-equality merge). By hypothesis, C_1 corresponds to walk W_1 and C_2 to walk W_2. The merged node C_m records the pair $\{W_1, W_2\}$ (Definition 1). Each walk uses only edges from E; the merge introduces no new edges, regardless of which of the four premise variants fired.
   The delta map of C_m is the component-wise sum $\delta_m = \delta_1 + \delta_2$: for each address v and token τ, $\delta_m[v, \tau] = \delta_1[v, \tau] + \delta_2[v, \tau]$. This is an arithmetic operation on values already derived from edges in E and introduces no new graph structure.
4) **Cycle connection (Algorithm 5, CONNECT-CYCLES).** Two chains C_1, C_2 at the same level with $s(C_1) = s(C_2)$ and complementary token flows ($\tau_{\text{out}}(C_1) = \tau_{\text{in}}(C_2)$) are connected when their combination satisfies $s(C_1 \cdot C_2) = d(C_1 \cdot C_2)$ (Definition 5). Two sub-cases arise.

Case (a): $d(C_1) = s(C_2)$ (address-adjacent at the junction). By hypothesis, C_1 corresponds to walk W_1 ending at $d(C_1)$ and C_2 to walk W_2 starting at $s(C_2) = d(C_1)$. Their concatenation $W_1 \cdot W_2$ is a valid walk by the same argument as Rule 1, and the closure condition makes it a cycle.

Case (b): both chains are cycles at the same vertex (R8/R9, MERGE-ADD). Both C_1 and C_2 are individually cycles at the shared vertex $v_0 = s(C_1) = d(C_1) = s(C_2) = d(C_2)$, each corresponding to a closed walk W_i in G using only edges from E. The merged node C_m records the concatenation $\mathit{transfers}(C_1) \cdot \mathit{transfers}(C_2)$, which forms a multi-walk $\{W_1, W_2\}$ (Definition 1) whose aggregate flow closes at v_0. Endpoint correspondence holds in the concatenated form: the first leaf of C_m has source v_0 (from W_1) and the last leaf has destination v_0 (from W_2). The delta map $\delta_1 + \delta_2$ captures the aggregate net flow. The closure-under-walk-sets lemma below verifies that this multi-walk representation satisfies Definition 5 at the economic level: each W_i closes individually, and the aggregate closes too. No new edges are introduced.

In both cases, every edge in the cycle node belongs to E.

Closure under walk sets. We verify that the set-of-walks representation used by merged and cycle-connected nodes (Definition 1) preserves the closure property needed by Theorem 3.

Lemma. *If a cycle node C with $s(C) = d(C)$ corresponds to a set of walks $\{W_1, \ldots, W_p\}$ per Definition 1, then the union $\bigcup_i W_i$ constitutes a closed flow at $s(C)$: for the traced token τ, the aggregate delta $\delta[s(C)] = \sum_i \delta_i[s(C)]$ equals the net inflow minus outflow at $s(C)$ across all walks.*

Proof. Each walk W_i uses only edges from E (by the inductive argument above). The delta map of the merged node is the component-wise sum $\delta = \sum_i \delta_i$. Since each δ_i is computed from the edges of W_i alone (amounts in minus amounts out at each address), the sum $\delta[s(C)]$ equals the total net inflow minus outflow at $s(C)$ across the union of all walks. The closure condition $s(C) = d(C)$ ensures that $s(C)$ is the common origin of every W_i and that the aggregate flow returns to $s(C)$. The token-match condition $\tau_{\text{in}} =_\tau \tau_{\text{out}}$ ensures that the inflow and outflow at $s(C)$ are denominated in equivalent tokens. Therefore $\delta[s(C)] > 0$ (verified by VALIDATE-DELTAS) means the aggregate return strictly exceeds the aggregate outlay at $s(C)$, satisfying the economic condition of Definition 5. □

Conclusion. The base case holds for T_0. Each of the four phases preserves walk correspondence. The closure lemma ensures that set-of-walks nodes satisfy Definition 5 when labeled as arbitrage. By induction on the number of rule applications, every chain in the final reduced AST corresponds to a walk (or set of walks) using only edges in G, and closed nodes faithfully represent arbitrage cycles. □

Lemma (Linearity). *Each* Leaf *is consumed by exactly one rule application and removed from its sibling list; no transfer edge appears in more than one chain in the reduced AST.*

Proof. Each rule in Table 1 consumes its operands and replaces them with a single output node; no rule duplicates a leaf or returns it to a sibling list. By induction on rule applications, leaves are partitioned across the chains of the reduced AST. □

Remark (multi-walk reading of Definition 5). The transfer chain $t_1, \ldots, t_k$ in Definition 5 denotes the *concatenated* leaf list of the chain node, which Theorem 1 decomposes into one or more walks in G: a singleton walk for chains built by sequential extension (R6/R10/R12), and a two-element walk-set for chains built by an additive merge (R8/R9). In both forms, the closure condition $s(t_1) = d(t_k)$ refers to the source of the first concatenated leaf and the destination of the last; endpoint correspondence (§1.3, mechanized boundary) ensures these coincide with the chain's structural endpoints $s(C)$ and $d(C)$. In the multi-walk form, each walk W_i is itself closed at the shared vertex v_0 (R8/R9 require both inputs to be cycles), so the aggregate flow returns to v_0 both leaf-wise and at the delta-map level. The token-match condition $\tau(t_1) =_\tau \tau(t_k)$ and the positivity condition $\delta[s(t_1)] > 0$ apply to the chain in either form. No extension of Definition 5 is needed: it covers single-walk cycles directly and multi-walk cycles via the concatenated leaf list.

1.2. Proof of Theorem 2 (Termination)

Proof. We exhibit a well-founded measure on ASTs and show that each non-trivial pass of ANNOTATE-AND-REDUCE (Algorithm 5) strictly decreases it.

Definitions. For an AST T, let:

- $u(T)$ = number of chain nodes in T not yet annotated (*chaining* or *merging*; chains labeled *cycle*, *arbitrage*, *token burn*, or *token mint* are considered labeled);
- $c(T)$ = total number of immediate children across all Tree nodes in T.

Define $\mu(T) = (u(T), c(T)) \in \mathbb{N} \times \mathbb{N}$, ordered lexicographically. This order is well-founded.

Label monotonicity (Lemma). We first establish that labels are monotonic. A chain receives its label from ANNOTATE-CYCLES: *cycle* when $s(C) = d(C)$, or *arbitrage* when additionally $\tau_{\text{in}} =_\tau \tau_{\text{out}}$, $\sigma(C) \notin C \vee s(C) = \sigma(C)$, and $\neg \mathit{WrapUnwrap}(C)$. Two monotonicity properties hold during the fixpoint loop:

- A labeled chain is never un-labeled. ANNOTATE-CYCLES only adds labels; it never removes them. CONNECT-CYCLES may consume labeled chains by merging them; the merged result is directly assigned a label (*arbitrage* or *cycle*) at merge time, so the number of unlabeled chains does not increase.

- The only label downgrade (*arbitrage* → *cycle*) occurs in VALIDATE-DELTAS, which runs *after* the fixpoint has converged and does not affect μ.

Therefore $u(T)$ is non-increasing across passes of ANNOTATE-AND-REDUCE.

Decrease per pass. Algorithm 5 applies two sub-steps to the current tree T:

1) $T' \leftarrow$ ANNOTATE-CYCLES$(T, from)$. Every chain C in T with $s(C) = d(C)$ that is not yet labeled receives a label. If at least one such chain exists: $u(T') < u(T)$. If none exists: $u(T') = u(T)$ and $T' = T$ structurally (annotation does not modify the tree topology, so $c(T') = c(T)$).
2) $T'' \leftarrow$ CONNECT-CYCLES$(T', from, to)$. This step scans for pairs of chains C_1, C_2 at the same level whose concatenation forms a cycle. Each successful connection replaces two children with one, so $c(T'') \leq c(T') - 1$ per merge. The merge consumes two chains and produces one. Crucially, the cycle-connection function (CONNECT-CYCLES) assigns the merged node a label (*arbitrage* or *cycle*) directly, based on the token-match and middleman conditions at merge time. Therefore: if both operands were labeled, the result is also labeled (u unchanged). CONNECT-CYCLES only selects pairs that are already labeled (*cycle*, *token burn*, or *token mint*), or specific *chaining* pairs in the ETH/WETH wrapping case; it never merges two unannotated chains. In all cases $u(T'') \leq u(T')$.

Case analysis. After both sub-steps, exactly one of three cases holds:

(i) $u(T') < u(T)$: at least one chain was newly labeled. Then $\mu(T'') \leq (u(T'), c(T'')) < (u(T), c(T)) = \mu(T)$ because the first component decreased.
(ii) $u(T') = u(T)$ and $c(T'') < c(T')$: no new labels, but at least one merge occurred. Then $\mu(T'') = (u(T), c(T'')) < (u(T), c(T)) = \mu(T)$ because the second component decreased while the first is unchanged.
(iii) $T'' = T$: neither sub-step changed the tree. The guard in Algorithm 5 returns T, and the recursion terminates.

In cases (i) and (ii), μ strictly decreases.

From lex-decrease to additive bound. Chains are created once in BUILD-AST, not in the fixpoint loop. From T_0 onward (the initial chained AST), both u and c are non-increasing: u by label monotonicity and by merging combining two unannotated chains into one; c by merging, lifting, and cycle-connection only removing or reshaping children without net increase. Hence lex-decrease of (u, c) coincides with a decrease of $u + c$ by at least one per pass, yielding the additive bound below.

Bound. Initially, $u(T_0) \leq n$ (at most one unlabeled chain per leaf, since chains arise from chaining leaves). The total children $c(T_0)$ equals the number of edges in the AST, which is $|T_0| - 1$. A tree with n leaves has at most $n - 1$ internal nodes (each internal node has ≥ 2 children), so $|T_0| \leq 2n - 1$ and $c(T_0) \leq 2n - 2$. Neither component ever increases across passes: u is non-increasing by label monotonicity, and c is non-increasing because merging only removes children. Since each non-trivial pass strictly decreases at least one component, it consumes at least one unit from the finite budget $u + c$. The recursion therefore terminates after at most $u(T_0) + c(T_0) \leq n + (2n - 2) = 3n - 2$ passes. The concrete $3n - 2$ bound is mechanized as `termination_bound` in the Rocq artifact, derived from `unlabeled_le_transfers` ($u \leq n$) and `cc_plus2_le_twice_ct` (handshaking, $c \leq 2n - 2$).

Practical bound. The bottom-up processing strategy (Section 3.2) means that leaf-level chaining completes in a single pass per tree level. The fixpoint in Algorithm 5 then operates on the already-flattened tree, where the number of remaining children is bounded by the number of independent paths. In practice the fixpoint converges in k passes, where k grows with the maximum width (sibling count) rather than the depth of the call hierarchy. □

1.3. Proof of Theorem 3 (Soundness)

Proof. We show that every transaction receiving the verdict *Arbitrage* from Algorithm 7 contains at least one subtree C in the reduced AST satisfying all conditions of Definition 5. The proof proceeds by contrapositive on each condition: we show that violating any condition of Definition 5 would trigger a verdict-determining reason in Algorithm 7, preventing the *Arbitrage* verdict.

Setup. Algorithm 7 accumulates a reason set R and assigns the verdict by strict priority:

1) If NO_CYCLES $\in R$: return *None*.
2) If LEFTOVERS $\in R$: return *Warning*.
3) If FINAL_NEG $\in R$: return *Warning*.
4) If FINAL_MIXED $\in R$: return *Warning*.
5) Otherwise: return *Arbitrage*.

The verdict is *Arbitrage* only when R contains none of NO_CYCLES, LEFTOVERS, FINAL_NEG, or FINAL_MIXED. We verify five conditions; note that all hold simultaneously when the verdict is *Arbitrage*, so we may use any condition's consequence when proving another.

Condition 1: Existence. NO_CYCLES is added to R when the set of arbitrage-labeled cycles $\mathcal{C}$ is empty (first check in Algorithm 7). Since NO_CYCLES $\notin R$, we have $\mathcal{C} \neq \emptyset$: at least one chain carries the *arbitrage* label.

Condition 2: Closure ($s(C) = d(C)$). ANNOTATE-CYCLES labels a chain as *cycle* only when $s(C) = d(C)$ (the closure check references Definition 5). Promotion to *arbitrage* preserves this property. CONNECT-CYCLES creates a cycle by concatenating chains $C_1 \cdot C_2$ only when $s(C_1 \cdot C_2) = d(C_1 \cdot C_2)$. Therefore every *arbitrage*-labeled chain satisfies $s(C) = d(C)$.

Condition 3: Token match ($\tau_{\text{in}} =_\tau \tau_{\text{out}}$). The promotion from *cycle* to *arbitrage* requires four conditions: (a) $s(C) = d(C)$ (already established), (b) $\tau_{\text{in}}(C) =_\tau$

$\tau_{\text{out}}(C)$, (c) $\sigma(C) \notin C$ or $s(C) = \sigma(C)$ (the cycle's σ does not appear as an intermediary, or is the cycle origin), and (d) $\neg\, \mathit{WrapUnwrap}(C)$ (the cycle is not a pure wrap/unwrap roundtrip). Chains satisfying only (a) retain the *cycle* label and are not counted in the arbitrage set C.

Condition 4: Positive net balance ($a_{out} > a_{in}$). This is enforced in two stages:

Stage 1: Gross balance. VALIDATE-DELTAS (Section 3.4) runs after the fixpoint converges. For each *arbitrage*-labeled chain C, it inspects $\delta[s(C)]$ for the traced token. If $\delta[s(C)] \leq 0$, the label is downgraded to *cycle*. Therefore every *arbitrage* chain surviving VALIDATE-DELTAS satisfies $\delta[s(C)] > 0$ at the gross level (before gas costs).

Stage 2: Net balance. Algorithm 7 computes the aggregate net balance Δ_{net} by subtracting gas costs (block-builder tip + base-fee burn) from the gross balance. We must show that the per-cycle gross positivity from Stage 1 implies aggregate net positivity.

Lemma (Delta decomposition). *When $L = \emptyset$ (no leftovers), the aggregate gross balance decomposes as $\Delta_{\text{gross}} = \sum_{C \in \mathcal{C}} \delta_C$ where $\mathcal{C}$ is the set of arbitrage-labeled cycles, and each $\delta_C = \delta[s(C)]$ is the per-cycle delta from Stage 1.*

Proof. When $L = \emptyset$, every transfer in the transaction is accounted for by a cycle in $\mathcal{C}$ or by the gas-cost transfers. By Theorem 1, each cycle's edges are original transfers from E, and by the Linearity Lemma above no transfer appears in two distinct cycles. The cycles therefore partition all non-cost transfers, and their deltas sum to the aggregate gross balance. □

By Stage 1, every $\delta_C > 0$. Therefore $\Delta_{\text{gross}} > 0$. The net balance is $\Delta_{\text{net}} = \Delta_{\text{gross}} - \mathit{gas}$. Two reasons guard the verdict: FINAL_NEG is added when Δ_{net} is negative, and FINAL_MIXED is added when it involves multiple tokens with opposing signs. Since neither reason is in R, every token component of Δ_{net} is strictly positive. Therefore $a_{out} > a_{in}$ after all costs.

Condition 5: Value-attribution completeness. LEFTOVERS is added to R when $L \neq \emptyset$ (leftover Leaf nodes not absorbed into any cycle or accounted for by gas costs). Since LEFTOVERS $\notin R$, we have $L = \emptyset$: every transfer in the transaction is accounted for. This is important because an unaccounted outgoing transfer from $s(C)$ could represent a hidden cost that invalidates the profit calculation. The absence of leftovers guarantees that the delta map faithfully reflects the true net balance of the cycle initiator.

Conversely, if leftovers are present ($L \neq \emptyset$), the reason LEFTOVERS is added and the cascade returns *Warning* before reaching the *Arbitrage* verdict. Therefore the *Arbitrage* verdict holds if and only if all five conditions are simultaneously satisfied: cycles exist ($C \neq \emptyset$), no leftovers remain ($L = \emptyset$), the gross delta is positive ($\Delta_{\text{gross}} > 0$), the net balance is positive ($\Delta_{\text{net}} > 0$), and it is not mixed.

Conclusion. By Theorem 1, every *arbitrage*-labeled chain corresponds to a walk or set of walks in the original transfer graph G, using only edges from E. Conditions 1–3 establish that the chain is a closed, token-matched cycle satisfying the structural requirements of Definition 5. Condition 4 establishes the economic requirement, and Condition 5 guarantees that the profit calculation is not invalidated by hidden flows. Therefore the verdict *Arbitrage* implies the existence of a Definition 5-compliant arbitrage cycle.

Remark. The converse does not hold: the algorithm may classify a genuine arbitrage as *Warning* when leftovers exist or the balance is mixed. Soundness guarantees the absence of false positives for the *Arbitrage* verdict; it does not guarantee the absence of false negatives.

Lemma (Delta consistency under $=_\tau$). *Let C be a cycle with $s(t_1) = d(t_k)$ and $\tau_{\text{in}} =_\tau \tau_{\text{out}}$, and let $N(\tau) = \{\tau' : \tau' =_\tau \tau\}$ denote the $=_\tau$-neighborhood of τ. The economically correct profit condition at $s(C)$ is $\sum_{\tau' \in N(\tau_{\text{in}})} \delta[s(C)][\tau'] > 0$. When $|N(\tau_{\text{in}})| > 1$, the per-token check $\delta[s(C)][\tau] > 0$ for a single τ underapproximates the aggregate.*

Proof. The cycle's net flow at $s(C)$ is the sum of incoming minus outgoing amounts across all transfers in C, per token. Under $=_\tau$, tokens in $N(\tau)$ represent the same economic asset; true profit is the sum across the neighborhood. When $\tau_{\text{in}} = \tau_{\text{out}}$ strictly, $N(\tau_{\text{in}}) = \{\tau_{\text{in}}\}$ and the sum reduces to the per-token check, which is what VALIDATE-DELTAS performs in the mechanized core. When $\tau_{\text{in}} \neq \tau_{\text{out}}$ but $=_\tau$-equivalent (e.g., ETH↔WETH), the sum is over the two-element neighborhood and is computed by the post-fixpoint promotion step, outside the mechanized core. Since $=_\tau$ is not transitive, N is not in general a partition; for the native↔wrapped pairs used in practice, $|N(\tau)| \leq 2$. □

Mechanized boundary. The verdict cascade of Algorithm 7 is captured in the mechanized core as `classify : list reason → verdict`, matching the same strict priority order (NO_CYCLES, LEFTOVERS, FINAL_NEG, FINAL_MIXED, *Arbitrage*). VALIDATE-DELTAS is captured by the predicate `validated_arbitrage`, which requires the chain to carry the *Arbitrage* label and a strictly positive gross delta at its origin for the traced token. The mechanized core establishes a single boundary statement, `classify_structural_soundness`, that exposes the structural shape of every cycle the algorithm surfaces with verdict *Arbitrage*. *Closure*: $s(C) = d(C)$, propagated from ANNOTATE-CYCLES, which only promotes a chain when its origin matches its destination, through the rewriting trace. *Endpoint correspondence*: the chain's structural origin and destination coincide with the source of its first leaf and the destination of its last leaf, so the syntactic endpoints of C are the actual transfer-graph endpoints of its underlying walk. *Walk decomposition*: the leaves of C decompose into a list of valid walks in G, a singleton walk-set for chains built by sequential extension (R6/R10/R12), and a two-element walk-set for chains incorporating a merge (R7/R8/R9/R13) where two parallel paths share endpoints. *Inclusion*: every leaf of C is an edge of the original transfer graph T_0,

so no edge is fabricated by the rewriting. *Positive gross delta*: the balance contribution of C at its origin is strictly positive for the traced token. The first four are inherited from `refinement_of_transfer_graph_cycles` and the walk-correspondence theorem (Theorem 1), both established by induction on the reflexive transitive closure of `rewrite_step` starting from a fresh initial CFT (no chains, only leaves and tree nodes from BUILD-AST). The last is the `validated_arbitrage` premise carried through unchanged. Together they discharge the structural shape of Definition 5 formally; the remaining net-delta condition (gross profit exceeds gas costs) is the delta-decomposition argument above.

□

1.4. Proof of Theorem 4 (Confluence)

Proof. We show that the reduced AST produced by the algorithm is uniquely determined by the input trace, regardless of implementation strategy. We decompose the rewrite system into its four sub-steps and prove that each either admits a unique outcome or is order-independent.

Sibling ordering (from Property 1). Sequential execution within each call frame means that siblings of a Tree node are ordered by trace position. In particular, a swap on an AMM produces two transfer events (one inbound, one outbound) within the same call frame, and they appear as consecutive siblings in the AST.

Step 1: Chaining is deterministic. The chaining procedure (Algorithm 4) processes the sibling list of each Tree node by a greedy scan: for each child c_i, it searches the remaining siblings $c_{i+1}, \ldots, c_m$ for the *first* (closest in trace order) compatible partner. We show this greedy choice is the only correct one.

We argue that for each child c_i, at most one compatible partner exists among its siblings, making the greedy choice the only one.

The key observation is that two transfers involving the same intermediary B are siblings in the AST only if they originate from the *same* call frame. In the EVM, each invocation of a pool contract is a separate CALL instruction that creates its own call frame. A swap at pool B produces exactly two transfer events (one inbound, one outbound) within that single call frame. If the arbitrage bot calls B twice (for two different swaps), each call creates a separate subtree in the AST; the events from the two calls are *not* siblings but belong to different Tree nodes.

We state this precisely.

Definition 2 (Swap event pair)**.** *Two sibling transfers ℓ_i, ℓ_j in a* Tree *node form a* swap event pair at address B *if $d(\ell_i) = B = s(\ell_j)$ (or vice versa), $\tau(\ell_i) \neq \tau(\ell_j)$, and both events were emitted within the same EVM call to B.*

Lemma (Swap-pair uniqueness). *Within any* Tree *node's sibling list, R1's premise ($d(\ell_i)=s(\ell_j)=B,\ \tau(\ell_i)\neq\tau(\ell_j)$) is satisfied by at most one sibling pair per (address, token-pair) combination $(B, \{\tau_i, \tau_j\})$.*

Proof. Each EVM CALL to contract B creates a separate call frame. The AST construction (Definition 2) places events from distinct call frames in distinct Tree subtrees. Therefore, all transfer events involving B that appear as siblings of the same Tree node originate from a single call to B. A standard AMM swap produces exactly one inbound and one outbound transfer event (the two legs of the swap), yielding one pair per address. Multi-token pools (e.g., Curve 3-pool, Balancer weighted pools) may emit more than two transfers per call, but Definition 2 requires $\tau(\ell_i) \neq \tau(\ell_j)$: each *token-qualified* pair (B, τ) appears at most once as source and once as destination, so swap-pair uniqueness holds per (address, token) combination. □

By this lemma, the chaining precondition ($d(c_i) = s(c_j)$, $\tau(c_i) \neq \tau(c_j)$) is satisfiable by at most one sibling per intermediary address and token pair. The greedy scan finds this unique partner (if it exists). For the transitive extension (chain + leaf or chain + chain), the same argument applies: the destination of the chain and the source of the next compatible node share an intermediary, and swap-pair uniqueness guarantees at most one match. Consequently, the greedy closest-first strategy produces the same result as any other selection strategy.

Step 2: Lifting is deterministic. A Tree node n is lifted (its children promoted to the parent) when all children of n are fully reduced (leaves or chains). This condition is structural: it depends only on the node types in the current tree, not on any ordering choice. Two observations:

- The set of liftable nodes is uniquely determined by the current tree state.
- Lifting two independent subtrees n_1, n_2 (neither is an ancestor of the other) commutes: lifting n_1 then n_2 produces the same tree as lifting n_2 then n_1, because they operate on disjoint parts of the tree.
- For nested subtrees, the bottom-up strategy lifts inner nodes first, which is the only valid order (a parent cannot be lifted until its children are reduced).

Therefore lifting produces a unique result.

Step 3: Merging and connection are order-independent. CONNECT-CYCLES performs two operations on chains at the same tree level: *parallel-path merging* (chains with the same origin and destination are combined) and *cycle connection* (chains with complementary token flows are concatenated to form cycles). We address each.

Parallel-path merging. When chains C_1, C_2 share the same origin and destination, they are merged into C_m. The merged node records the union of walks and the component-wise sum of delta maps. Both operations are associative and commutative:

$$(W_1 \cup W_2) \cup W_3 = W_1 \cup (W_2 \cup W_3)$$

$$(\delta_1 + \delta_2) + \delta_3 = \delta_1 + (\delta_2 + \delta_3)$$

Therefore, when k chains share endpoints, the final merged node is the same regardless of pairing order.

Cycle connection. Two chains C_1, C_2 with complementary token flows are concatenated when the result closes. Ambiguity could arise if a chain C_1 is compatible with C_2 (for connection) and also with C_3 (for merging). However, the fixpoint loop (Algorithm 5) ensures convergence regardless of per-pass ordering. Suppose one pass chains C_1 with C_2 and leaves C_3 separate; a later pass then merges the result with C_3 (or vice versa). Alternatively, if the first pass merges C_1 with C_3, a later pass chains the merged node with C_2. In both scenarios, the fixpoint includes the same set of edges and the same delta map: chaining extends a path by concatenation, and merging aggregates parallel paths by union. The final normal form (after convergence) contains all transfers from C_1, C_2, and C_3 regardless of intermediate ordering, because every applicable rule eventually fires during the fixpoint iteration. Formally, every edge consumed by a merge or connection in one ordering is also consumed in any other ordering before the fixpoint terminates (by Theorem 2, termination is guaranteed, and the fixpoint guard exits only when no further rules apply).

Step 4: Annotation is deterministic. ANNOTATE-CYCLES labels each chain based on structural properties: closure ($s(C) = d(C)$), token match ($\tau_{\text{in}} =_\tau \tau_{\text{out}}$), intermediary membership ($\sigma(C) \notin C$ or $s(C) = \sigma(C)$), and the *WrapUnwrap* guard (R14 only). These are pure predicates on the chain's attributes, independent of any ordering. $\sigma(C)$ is itself a total function of the trace (the σ cascade selects a unique value when the chain's transfer set admits two valid orientations). Given the same chain, the label is always the same.

Composition of the fixpoint loop. Algorithm 5 applies two sub-steps per pass: first ANNOTATE-CYCLES (Step 4), then CONNECT-CYCLES (Step 3). By Theorem 2, the loop terminates after finitely many passes. We show the sequence of intermediate trees is uniquely determined.

Let T be the input to a pass. ANNOTATE-CYCLES produces T' by labeling chains; this is deterministic (Step 4), so T' is uniquely determined by T. CONNECT-CYCLES produces T'' from T' by merging and connecting chains; this is order-independent (Step 3), so T'' is uniquely determined by T'. The fixpoint guard checks $T'' = T$; since T'' is unique, the guard's outcome is deterministic.

By induction on the pass number: if T_i is uniquely determined, then T_{i+1} is uniquely determined. The initial tree T_0 (output of leaf manipulation) is unique by Steps 1–2. Therefore the entire sequence $T_0, T_1, \ldots, T_k$ is unique.

Full pipeline. The complete algorithm applies:

1) BUILD-AST: deterministic (trace $\to$ AST is a function).
2) Leaf manipulation (Algorithm 4): chaining is deterministic (Step 1) and lifting is deterministic (Step 2).
3) Node manipulation (Algorithm 5): the fixpoint is unique (composition above).
4) VALIDATE-DELTAS: deterministic (pure predicate on delta maps).
5) Classification (Algorithm 7): deterministic (priority cascade on reason set).

Each stage produces a unique output from its input. Therefore the final reduced AST and the verdict are uniquely determined by the input execution trace.

Mechanized proof. In the Rocq mechanization, the step is realized as a computable function whose determinism is immediate from the function definition. Confluence then follows by induction on the derivation length: if $T_0 \to^* T_f$ and $T_0 \to^* T'_f$ with both T_f, T'_f normal forms, determinism forces $T_f = T'_f$. Combined with the constructively-proved well-foundedness of the measure μ, this yields the uniqueness corollary $\exists! \, T_f.\ T_0 \to^* T_f$. □

Remarks. *Critical pairs absent Property 1.* Without the canonical traversal order Property 1 provides, overlapping premises across rule pairs would form critical pairs. Two illustrative cases. *(i) R6 vs R10.* R6 extends a chain with a leaf and R10 composes two chains. Given a chain C and an adjacent leaf-pair (ℓ_1, ℓ_2) that already chains as C', the same triple (C, ℓ_1, ℓ_2) admits two reductions: $C + \ell_1$ first then composing with ℓ_2 via R6+R6, or $\ell_1 + \ell_2 = C'$ first then composing $C + C'$ via R10. The two paths produce semantically equal chains but distinct intermediate trees, so classical Knuth–Bendix would require a critical-pair joinability proof. *(ii) R7 vs R9 on a closed pair.* R7 merges parallel chains with distinct τ_{in} and shared τ_{out}; R9 merges closed cycles with shared τ_{in}. When two chains satisfy both ($s = d$ on both, all four leg tokens distinct in pairs), either rule could fire, and the two merges yield identical multisets but different tree shapes if either rule were applied non-deterministically. With Property 1 in place, the deterministic step function selects exactly one rule per redex by trace order; no critical pair arises in the rewriting relation we mechanize. *Routers parametricity.* Theorem 4 holds for any fixed instantiation of `is_singleton_router`: different Routers sets induce different normal forms, but the convergence and uniqueness theorems are insensitive to the choice.

1.5. Proof of Theorem 5 (Decidable equivalence)

Proof. By Theorem 2, every term reaches a normal form in at most $3n - 2$ steps, where n is the number of leaves. By Theorem 4, the rewrite relation is confluent. A terminating and confluent system is convergent, so every term has a unique normal form $T{\downarrow}$ (Corollary 1).

It remains to show the equivalence. Suppose $T \equiv_R T'$, i.e. T and T' are joinable: $\exists\, U.\ T \to^* U \leftarrow^* T'$. By confluence, U can be further reduced to $T{\downarrow}$ and to $T'{\downarrow}$; uniqueness gives $T{\downarrow} = T'{\downarrow}$. Conversely, if $T{\downarrow} = T'{\downarrow}$ then $T \to^* T{\downarrow} \leftarrow^* T'$, so T and T' are joinable.

Since computing $T{\downarrow}$ terminates and syntactic equality of trees is decidable, the word problem for $\equiv_R$ is decidable. □

By Church-Rosser (confluence + termination), joinability coincides with convertibility; $\equiv_R$ is equivalently defined either way.

Proposition (Refinement of transfer-graph cycles). *Let G be the transfer graph of a transaction tx (Definition 1). Every chain in $T\downarrow$ labeled arbitrage forms a closed walk in G using only edges of G. The inclusion is strict.*

Proof. By Theorem 1, each chain in $T\downarrow$ corresponds to a walk in G using only edges of G. R14 fires only on chains with $s(C) = d(C)$ at the structural level; via endpoint correspondence (§C.3, mechanized boundary), the chain's structural endpoints coincide with the transfer endpoints $s(t_1)$ and $d(t_k)$, so the corresponding walk is closed. Strict inclusion is witnessed by Appendix D.2 (yield harvest): a transaction with eight closed walks in G none of which satisfies R14's $\tau_{\text{in}} =_\tau \tau_{\text{out}}$ condition. □

1.6. Rocq Trust Base

The development declares 13 `Parameters`. None is used as an axiom about the rewriting system; every `Parameter` is either an opaque type (with decidable equality) or a boolean predicate case-split in the proofs. The theorems therefore hold for any extraction instantiation.

- `address : Type` and `address_eq_dec`: opaque address type with decidable equality;
- `token : Type` and `token_eq_dec`: opaque token type with decidable equality;
- `token_equiv : token -> token -> bool`: case-split in the proofs, so soundness holds for any boolean instantiation. A `Prop` predicate `is_token_equiv_well_formed` (reflexivity and symmetry; transitivity not required because $=_\tau$ is checked only at cycle boundaries) is a deployment well-formedness obligation, not a theorem precondition. Recall scales with the chosen relation; soundness is invariant.
- `is_burn`, `is_mint`, `is_singleton_router`, `is_token_contract`: four boolean predicates case-split in the proofs. `is_token_contract` is used by the `wrap_unwrap` fixpoint guarding R14.
- `net_positive : chain_tree -> bool`: the deployer's cost model. Realizer choices: gross delta exceeds gas plus block-builder payment on Ethereum; plus L1-data cost on Arbitrum; chain-specific basis on BSC. The predicate is case-split in `validate_deltas` and threaded through `validated_arbitrage`, so the five theorems are parametric in the cost model and transfer to any chain whose realizer is supplied. The attempted-arbitrage population is formally characterized as the structurally-Def-5 cycles for which `net_positive` returns `false` (Corollary `attempted_arbitrage_characterization` in `Arbitrage.v`).
- `merge_match_holds`, `merge_match_closure`, `merge_match_token`: deployment trust that the kernel's `chains_mergeable` fires only on chain pairs satisfying one of R7/R8/R9/R13's structural premises, with the merged chain closed under τ-equivalence. Maintained by the operational choice of which chains the kernel attempts to merge; closes the Phase-3 bridge (`try_merge_children_to_rewrite_star`) unconditionally.

At extraction, the opaque types are instantiated with the concrete OCaml types used by the implementation; `is_burn` and `is_mint` are instantiated via standard event-signature matching, and `is_singleton_router` uses the optional *Routers* address set (§3).

Phase-3 bridge from kernel to rules.. The executable kernel's Phase-3 pass (`step_fn`: annotate every closed chain, then merge one pair) is realized by a sequence of declarative `rewrite_step` applications. The bridge proceeds in two stages.

Annotate phase. `annotate_all_fn_to_rewrite_star` proves that, on any `RTree` with flat children, the annotation pass corresponds to a sequence of `RS_annotate_arb` / `RS_annotate_cyc` applications. The proof is a left-to-right cascade (`annotate_cascade`) over the children list, using the rules' left- and right-context form (L ++ `[RChain` c`]` ++ R) to target any position in the list. The function was tightened to enforce the R14 premise `wrap_unwrap` c `= false`, matching Table **??**.

Merge phase. The four merge rules (R7/R8/R9/R13) are stated with a middle context (L ++ `[`c_1`]` ++ M ++ `[`c_2`]` ++ R), which models the non-adjacent merge produced by the kernel's scanning combiner. The bridge introduces `merge_match` as the structural premise capturing the disjunction of R7/R8/R9/R13's token preconditions, and proves `merge_match_to_rewrite_step`: under `merge_match`, the function-emitted merged chain is reachable by exactly one `RS_merge_`$\star$ step (with the merged chain in its declarative `Merging`-labeled form). A subsequent `annotate_after_merge` step relabels `Merging` to `Arbitrage` or `Cycle`, matching the kernel's `merge_two_chains` output via either `RS_annotate_arb` or `RS_annotate_cyc`.

`step_fn_witness_to_rewrite_star` composes the two stages. The witness predicate `step_fn_witness` states exactly the structural invariant the kernel must maintain for each Phase-3 step. Maintaining the invariant is a deployment obligation, analogous to the `net_positive` cost-model realizer: it is discharged by the operational choice of which chains the kernel attempts to merge, and is empirically validated by the cross-chain bench in Appendix **??**. The bridge therefore gives `rewrite_star` coverage of every `step_fn`-produced reduced CFT, modulo this trust assumption stated as a single `Prop` (`merge_match`), not an `Axiom`.

2. Structural Equivalence in Practice

Theorem 5 states that two CFTs are equivalent iff their canonical forms coincide. This appendix develops the practical consequences of this result in three steps: we first build intuition through a concrete example (Section 2.1),

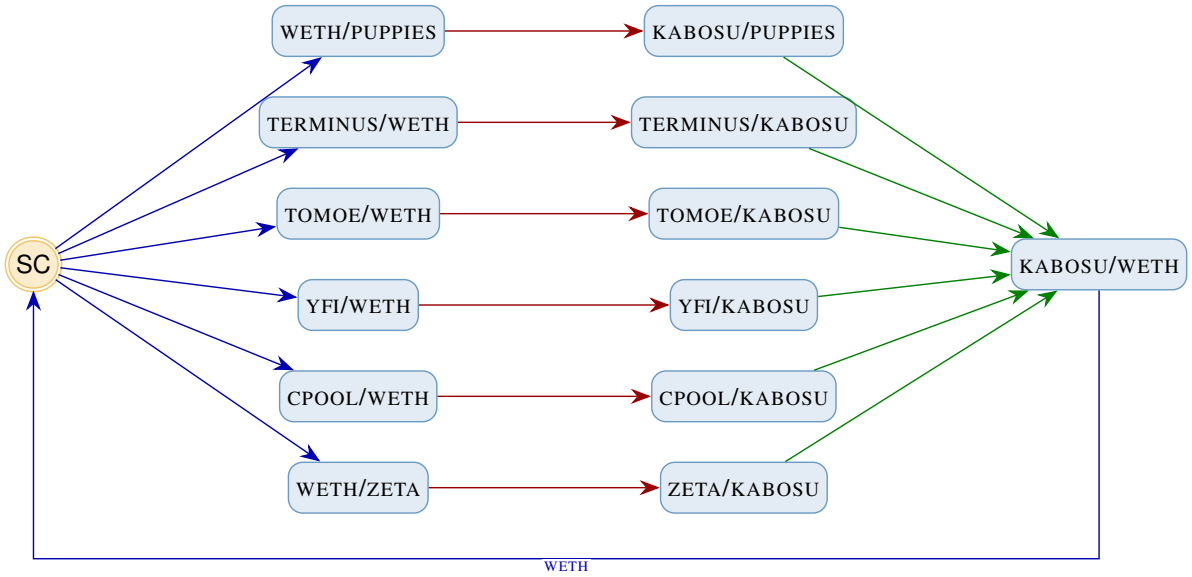

Figure 1: Transfer graph of tx_S. Six parallel paths through twelve pools, same star topology as tx_R (Figure 2).

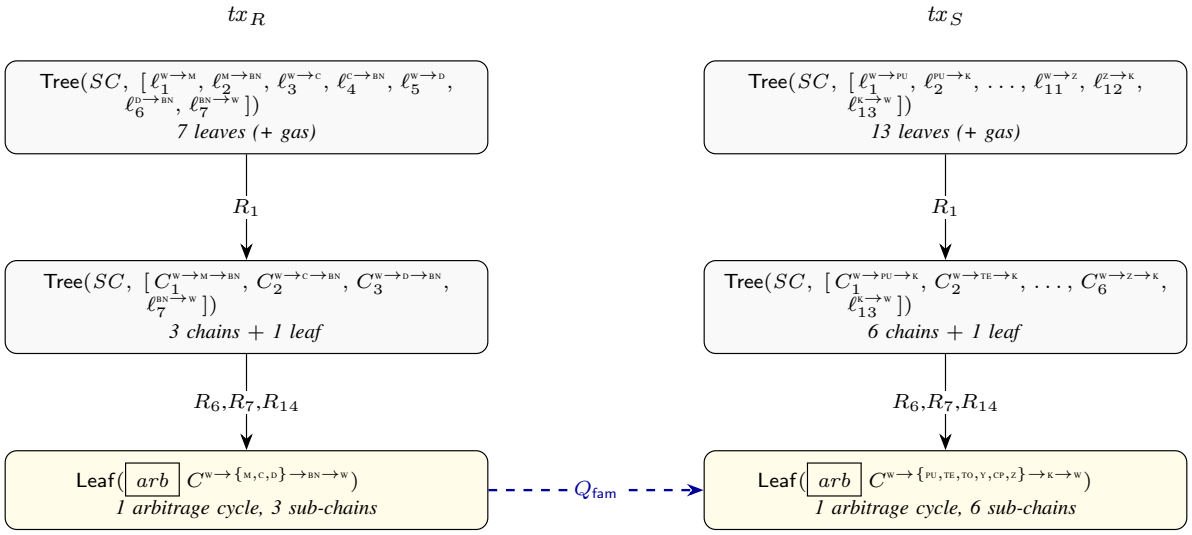

Figure 2: Side-by-side reduction of tx_R and tx_S. Despite different tokens and arities (3 vs. 6), both reduce to a single arbitrage cycle with N merged sub-chains. The dashed arrow marks family equivalence (Q_{fam}, Section 2.4): different normal forms, same topology.

then formalize the notion of strategy family via a context-free tree grammar (Section 2.2), and finally sketch a query language over normal forms (Section 2.4).

2.1. Structural Equivalence by Example

We illustrate structural equivalence on two confirmed arbitrages from the evaluation dataset. The two transactions involve entirely different tokens, different pool contracts, and a different number of internal swaps, yet their reductions converge to the same topological shape.

tx_R.. The running example from Section 3 (Figure 2). A smart contract receives 0.0575 WETH and splits it into three parallel swap paths: one through a WETH/MOON pool, one through CHAD/WETH, and one through DUCK/WETH. Each path converts its intermediate token (MOON, CHAD, DUCK) into BEAN at a dedicated second-hop pool. All three BEAN streams converge on a single BEAN/WETH pool, which returns 0.0598 WETH to the smart contract. The raw trace contains 10 transfer events across 6 pool interactions and produces 16 reduction steps.

tx_S.. [1]A different smart contract executes the same strategy with more paths (Figure 1). It splits WETH into

1. 0xaa4b2aec9c7a571f96062576135632ece91bde142075963219b26232404dfc5c

six parallel paths through PUPPIES, TERMINUS, TOMOE, YFI, CPOOL, and ZETA. Each path converts its intermediate token into KABOSU at a dedicated pool, and all six KABOSU streams converge on a single KABOSU/WETH pool. The raw trace contains 19 transfer events across 12 pool interactions.

Reduction (Figure 2).. Despite their different sizes, both transactions undergo the same reduction sequence. First, leaf chaining (R_1) pairs the inbound and outbound transfers within each pool call frame. After lifting, chain extension (R_6) connects per-pool chains into N multi-hop paths (N=3 for tx_R, N=6 for tx_S). Then, merge (R_7) combines the N paths that share the same origin and destination into a single compound chain, and a final chaining (R_6) appends the return leg. Annotation (R_{14}) recognizes the closed cycle and labels it as an arbitrage.

The canonical forms differ only in arity (3 vs. 6 subchains) and in the concrete token names. Abstracting over both, $tx_R{\downarrow}$ and $tx_S{\downarrow}$ instantiate the same parametric shape:

$$\text{WETH} \xrightarrow{N \text{ paths}} \tau_i \;\rightarrow\; \text{TARGET} \;\rightarrow\; \text{WETH}$$

By Theorem 5, this structural identity is decidable: reduce both CFTs and compare. We chose a simple example where the equivalence is visually obvious. The value of the algebra is that the same mechanical procedure, reduce and compare, decides equivalence for any pair of words derivable from the rewriting system, including transactions whose raw traces contain hundreds of transfers and nested contract calls where visual assessment is impossible.

Implications.. The canonical form opens several directions beyond detection. A system that indexes reduced CFTs could retrieve *all* star-shaped convergent arbitrages from a historical database in a single structural query, regardless of which tokens or pools were used. Normal forms also provide a natural feature space for unsupervised learning: clustering on normal-form trees groups transactions by fund flow topology, automatically discovering strategy families without labeled training data. Because the canonical form is unique (Theorem 5), two transactions in the same cluster are guaranteed to share the same structural semantics.

The equational theory also induces a natural *taxonomy*. At the finest level, two transactions belong to the same equivalence class iff their canonical forms are syntactically identical; tx_R (N=3) and tx_S (N=6) fall in different classes. At a coarser level, one can quotient over arity and token identities to obtain *parametric families* where both belong to the same family (star-convergent arbitrage with variable N). The following sections formalize this hierarchy.

2.2. A Context-Free Grammar for Strategy Families

The parametric shape shared by tx_R and tx_S can be described by a family of context-free grammars. We introduce two constructors over canonical forms: $\mathsf{Star}(\alpha, L, \alpha)$ denotes an arbitrage-annotated cycle (R14) with initiator token α, and $\mathsf{Swap}(\tau, \tau')$ denotes a transfer chain with entry token τ and exit token τ'. We write $\circ$ for sequential chaining (within an arm) and $\|$ for parallel composition (across arms). Let Σ be a countably infinite set of token identifiers. For each pair $(\alpha, \tau') \in \Sigma^2$, define the grammar $G_{\alpha,\tau'} = (\{S, L, C, H\},\ \Sigma,\ P_{\alpha,\tau'},\ S)$ where the production set $P_{\alpha,\tau'}$ is:

$$\begin{aligned}
S &\rightarrow \mathsf{Star}(\alpha,\ L,\ \alpha) \\
L &\rightarrow C \parallel L \mid C \\
C &\rightarrow \mathsf{Swap}(\alpha, \tau_1) \circ H_{\tau_1} \\
H_\tau &\rightarrow \mathsf{Swap}(\tau, \tau') \mid \mathsf{Swap}(\tau, \tau_k) \circ H_{\tau_k} \quad \text{for any } \tau_1, \tau_k \in \Sigma
\end{aligned}$$

The *star-convergent family* is the language $\mathcal{L}_{\mathsf{star}} = \bigcup L(G_{\alpha,\tau'})$ ranging over $(\alpha, \tau') \in \Sigma^2$. Each individual grammar $G_{\alpha,\tau'}$ is context-free: the start symbol determines α (the initiator token), the production for C binds the convergence token τ' across all branches, and the intermediate tokens τ_1, τ_k are freely chosen per cycle. Strictly, H_τ is a family of nonterminals indexed by τ; for any finite transaction, only finitely many are reachable. The recursive production H_τ allows swap paths of arbitrary length: a two-hop path uses $H_\tau \rightarrow \mathsf{Swap}(\tau, \tau')$, a three-hop path uses $H_\tau \rightarrow \mathsf{Swap}(\tau, \tau_k) \circ H_{\tau_k}$ with one further expansion, and so on. A bottom-up ranked tree automaton cannot enforce that all siblings share the same τ', since sibling states are computed independently. For finite Σ, an unranked hedge automaton could encode τ' in its state space; over countable Σ (the general case) this is not possible, and the constraint is properly context-free. In either case, the grammar is the more natural and compositional specification.

Remark. In practice, Σ is finite: the number of deployed token contracts on any chain is finite, and the gas limit of a transaction bounds the number of transfers (and hence distinct tokens) in any single canonical form. The arity N of a derivation is similarly bounded. The grammar is defined over a countable alphabet for generality, but every concrete word has bounded depth (Theorem 2) and bounded width (gas). The grammar is extensible: new strategy families (e.g., flash-loan-wrapped arbitrages, sequential compositions) require only new productions in S, not changes to the rewriting system.

Recall the skeleton map sk (§3.7) is an α-abstraction; the grammar $G_{\alpha,\tau'}$ enumerates its orbits parametrically. The rewriting rules R1–R15 inspect only the structural predicates (`is_burn`, `is_mint`, `is_singleton_router`, $=_\tau$), so for any renaming π preserving them, $\mathsf{sk}(\pi(T{\downarrow})) = \pi(\mathsf{sk}(T{\downarrow}))$. The Rocq mechanization makes this parametricity explicit: address and token are declared as opaque types.

Proposition 1 (Grammar characterization)**.** *Let $T{\downarrow}$ be a normal form containing an arbitrage cycle (Definition 5) with hub address h, entry/exit token α, and one or more parallel swap branches converging at a common intermediate token τ'. Then $T{\downarrow} \in L(G_{\alpha,\tau'})$. Conversely, every word in $L(G_{\alpha,\tau'})$ is the structural image of such a star-convergent arbitrage.*

Sketch. ($\Rightarrow$) By termination and confluence (Theorems 2 and 4), the chaining and lifting rules (R1, R6) collapse each branch into a swap sequence $\mathsf{Swap}(\alpha, \tau_1) \circ \cdots \circ$

$\mathsf{Swap}(\tau_{k-1}, \tau')$, matching $C \to \mathsf{Swap}(\alpha, \tau_1) \circ H_{\tau_1}$ with H unfolded as needed (finite by termination). Parallel sibling branches at the hub compose via $L \to C \parallel L$. The outer $\mathsf{Star}(\alpha, L, \alpha)$ records that branches return to hub h's account in token α, the closure witnessed by ANNOTATE-CYCLES (R14). ($\Leftarrow$) Any derivation in $G_{\alpha,\tau'}$ exhibits a Star node whose branches route tokens through τ' and return to α; this is a valid CFT normal form by construction, and R14 annotates it as an arbitrage when $\sigma(C) \notin C \vee s(C) = \sigma(C)$ and $\neg\, \mathit{WrapUnwrap}(C)$, with $\delta > 0$ checked externally by VALIDATE-DELTAS. □

The canonical form of tx_R (N=3) is:

$$\begin{aligned} tx_R{\downarrow} \;=\; \mathsf{Star}(\mathrm{weth},\, &(\mathsf{Swap}(\mathrm{weth}, \mathrm{moon}) \circ \mathsf{Swap}(\mathrm{moon}, \mathrm{bean})) \\ &\parallel (\mathsf{Swap}(\mathrm{weth}, \mathrm{chad}) \circ \mathsf{Swap}(\mathrm{chad}, \mathrm{bean})) \\ &\parallel (\mathsf{Swap}(\mathrm{weth}, \mathrm{duck}) \circ \mathsf{Swap}(\mathrm{duck}, \mathrm{bean})),\ \mathrm{weth}) \end{aligned}$$

The canonical form of tx_S (N=6) is:

$$\begin{aligned} tx_S{\downarrow} \;=\; \mathsf{Star}(\mathrm{weth},\, &(\mathsf{Swap}(\mathrm{weth}, \mathrm{puppies}) \circ \mathsf{Swap}(\mathrm{puppies}, \mathrm{kabosu})) \\ &\parallel (\mathsf{Swap}(\mathrm{weth}, \mathrm{terminus}) \circ \mathsf{Swap}(\mathrm{terminus}, \mathrm{kabosu})) \\ &\parallel (\mathsf{Swap}(\mathrm{weth}, \mathrm{tomoe}) \circ \mathsf{Swap}(\mathrm{tomoe}, \mathrm{kabosu})) \\ &\parallel (\mathsf{Swap}(\mathrm{weth}, \mathrm{yfi}) \circ \mathsf{Swap}(\mathrm{yfi}, \mathrm{kabosu})) \\ &\parallel (\mathsf{Swap}(\mathrm{weth}, \mathrm{cpool}) \circ \mathsf{Swap}(\mathrm{cpool}, \mathrm{kabosu})) \\ &\parallel (\mathsf{Swap}(\mathrm{weth}, \mathrm{zeta}) \circ \mathsf{Swap}(\mathrm{zeta}, \mathrm{kabosu})),\ \mathrm{weth}) \end{aligned}$$

Both derivations belong to $\mathcal{L}_{\mathsf{star}}$: $tx_R{\downarrow} \in L(G_{\mathrm{weth,bean}})$ with N=3, and $tx_S{\downarrow} \in L(G_{\mathrm{weth,kabosu}})$ with N=6. Any star-convergent arbitrage belongs to $\mathcal{L}_{\mathsf{star}}$ for the appropriate (α, τ'): a 10-arm star through ten different memecoins, a 2-arm path through stablecoins, or a star with three-hop swap chains through nested intermediaries. The grammar generates an infinite family; tx_R and tx_S are two concrete members.

Structural equivalence at the exact level ($\equiv_R$) distinguishes them because their canonical forms differ in arity and token names. Family equivalence (Q_{fam} in Section 2.4) identifies them: any two words in $\mathcal{L}_{\mathsf{star}}$ belong to the same strategy family, regardless of how many arms they have or which tokens they route through.

2.3. Extending the Grammar: Flash-Loan Wrappers

The grammar is extensible without modifying the rewriting system. We illustrate this by adding productions that capture flash-loan-wrapped arbitrages, the family that contains 63% of our exclusive confirmed detections (Category 2, §4.3). The extension is instructive because it forces us to reason about what survives the fixpoint.

What the rewriting preserves.. A flash loan consists of two transfers: the borrow $\ell_b = (\mathrm{pool}, \mathrm{bot}, a, \tau_\ell)$ and the repay $\ell_r = (\mathrm{bot}, \mathrm{pool}, a, \tau_\ell)$, with identical amount, identical token, and reversed endpoints. These form a specular pair in the sense of §3.4. The leftover recovery pass removes the pair from the leaf list and emits a *leftover cycle*, a first-class annotated node that records the loan token and amount. Leftover cycles are preserved by every subsequent rewrite rule (they are neither chained, lifted, nor merged with ordinary chains), so they survive into the canonical form. The arbitrage cycle that *uses* the borrowed capital reduces independently under the standard star-convergent rules; the two appear side by side in the canonical form.

Grammar extension.. A canonical form is a pair (N, Λ) where N is a star-convergent derivation and Λ is a (possibly empty) sequence of leftover cycles whose token coincides with the star's hub. For each $(\alpha, \tau') \in \Sigma^2$, define $G^F_{\alpha,\tau'} = (\{F, N, L, C, H, \Lambda\}, \Sigma, P^F_{\alpha,\tau'}, F)$ with productions:

$$\begin{aligned} F &\to (N,\ \Lambda) \\ N &\to \mathsf{Star}(\alpha,\ L,\ \alpha) \\ L &\to C \parallel L \mid C \\ C &\to \mathsf{Swap}(\alpha, \tau_1) \circ H_{\tau_1} \\ H_\tau &\to \mathsf{Swap}(\tau, \tau') \mid \mathsf{Swap}(\tau, \tau_k) \circ H_{\tau_k} \\ \Lambda &\to \varepsilon \mid \mathsf{LC}(\alpha, a) \cdot \Lambda \end{aligned}$$

As in §2.2, H_τ is a family of nonterminals indexed by τ. When $\Lambda = \varepsilon$, $G^F_{\alpha,\tau'}$ reduces to the star grammar $G_{\alpha,\tau'}$; when Λ is non-empty, the LC tokens coincide with the hub α, capturing flash-loan wrappers on the cycle's entry token. The extension thus subsumes pure stars and adds flash-loan-wrapped arbitrages without modifying the rewriting system. The productions generate all (possibly empty) sequences of $\mathsf{LC}(\alpha, \cdot)$; we define $L(G^F_{\alpha,\tau'})$ as the image after canonicalizing Λ to its sorted representative (ascending amount). Over the practical finite alphabet of amounts, $L(G^F_{\alpha,\tau'})$ is context-free and membership is decidable in linear time; queries below treat Λ as a set ($\mathsf{LC} \in \Lambda$). The total star-extended family is

$$\mathcal{L}_{\mathsf{arb}} \;=\; \bigcup_{(\alpha,\tau') \in \Sigma^2} L(G^F_{\alpha,\tau'}),$$

and the flash-loan family $\mathcal{L}_{\mathsf{flash}} \subset \mathcal{L}_{\mathsf{arb}}$ is the subset with Λ non-empty. Specular pairs on tokens unrelated to the cycle (a refund, a meta-transaction relay) do not satisfy any $G^F_{\alpha,\tau'}$ and are therefore outside the family. Membership is decidable in linear time in the size of the canonical form.

The 17 912 WETH arbitrage as a concrete member.. We illustrate on Ethereum transaction `0x45388b0f9ff46ffe98a3124c22ab1db2b1764ecb3b61234e29e5c9732b7fd4ab`, a confirmed arbitrage from the exclusive detections that Eigenphi does not flag (full forensic analysis in Appendix D.1). A bot borrows 14 175.71 WETH from an aggregator, executes a three-hop cycle through Bancor, a BNT/AAVE pool, and an AAVE/WETH pool, and repays the loan.

Figure 3 shows the full decoded transfer graph, with the detected star-convergent cycle highlighted in green and the flash-loan specular pair in dashed red; remaining transfers (wrap/unwrap, builder bribe, EOA invocation) are shown in grey.

After reduction, the main arbitrage cycle annotates under R14; the borrow/repay pair produces a leftover cycle under

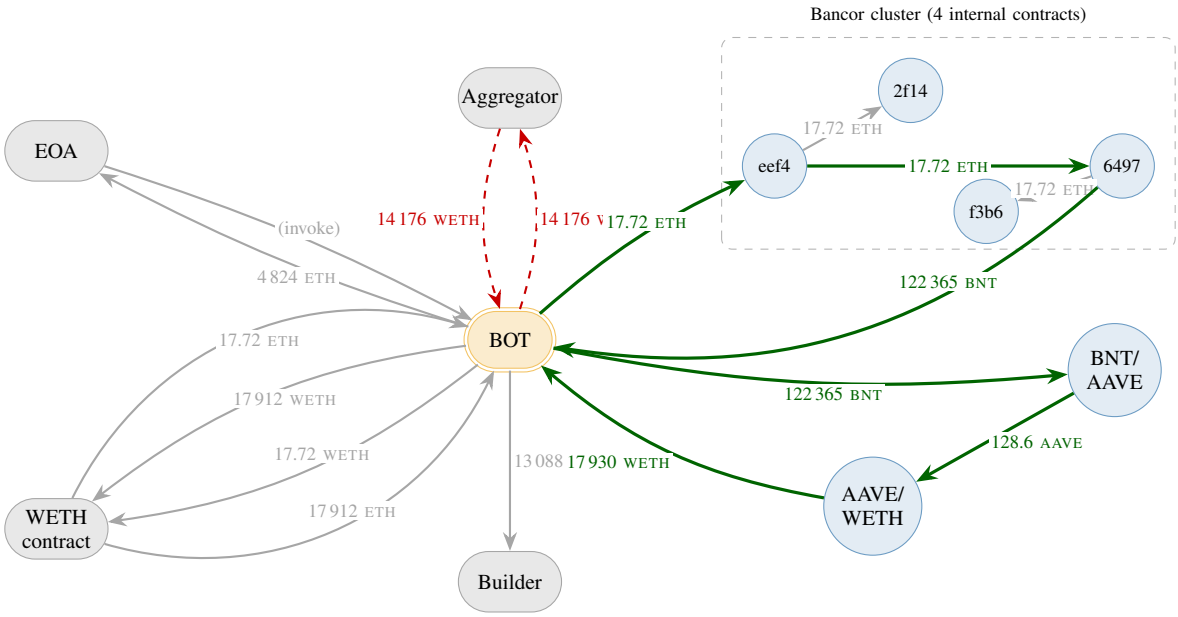

Figure 3: Full transfer graph of the 17 912 WETH arbitrage, all 14 transfers in the decoded trace. Bancor comprises four internal contracts (labels show the first four hex characters of each address) with three internal ETH hops that the algorithm routes through. **Green**: the star-convergent cycle our system extracts ($\mathsf{Star}(\text{weth}, S_{\mathsf{BAW}}, \text{weth})$). **Dashed red**: the flash-loan specular pair, collapsed by leftover recovery into $\mathsf{LC}(\text{weth}, 14\,176)$. Grey: the remaining transfers (EOA invocation, two wrap/unwrap pairs, builder bribe, profit return). The narrative in §D.1 flattens the Bancor cluster into a single node.

the specular rule; the profit unwrap is a second annotated cycle on WETH (native/wrapped equivalence). Writing

$$S_{\mathsf{BAW}} = \mathsf{Swap}(\text{weth}, \text{bnt}) \circ \mathsf{Swap}(\text{bnt}, \text{aave}) \circ \mathsf{Swap}(\text{aave}, \text{weth})$$

for the three-hop arm and ignoring the profit unwrap (a separate star on WETH that contributes no additional content to the family-membership test), the relevant normal-form pair is

$$(N, \Lambda) = \big(\mathsf{Star}(\text{weth}, S_{\mathsf{BAW}}, \text{weth}), \mathsf{LC}(\text{weth}, 14\,175.71)\big),$$

a word in $L(G^F_{\text{weth,weth}})$ (here $\alpha = \tau' = \text{weth}$), hence in $\mathcal{L}_{\mathsf{flash}}$. Only genuine borrow/repay wrappers on the cycle's entry token qualify; unrelated specular pairs do not.

Decidability lifts directly.. The family-membership query for $\mathcal{L}_{\mathsf{flash}}$ decomposes into (i) parsing $tx\downarrow$ as a (N, Λ) pair, which is a linear scan of the canonical output, and (ii) checking the token-match condition, which is a set comparison on the leftover cycles. Both are decidable in polynomial time. Corollary 1 (uniqueness of normal forms) guarantees that the answer is independent of the reduction order. The same decomposition handles other specular-pair wrappers (meta-transaction relays, approval refunds) by varying the matching predicate; the grammar skeleton does not change.

2.4. From Detection to Query Language

The convergent rewriting system defines more than a detection algorithm: it induces a decidable equational theory over fund flow trees. We sketch how this theory gives rise to a query language for transaction forensics, and illustrate each query on the running examples tx_R and tx_S.

Normal forms as an intermediate representation.. Every transaction t reduces to a unique normal form $t\downarrow$ (Corollary 1) whose shape encodes the fund flow structure: cycles, chains, leftovers, and their nesting. The rewriting system acts as a compiler from raw execution traces to a canonical intermediate representation. Any *structural* question about a transaction reduces to a predicate over its canonical form, and because the representation is unique, equivalent queries on equivalent traces always agree.

Formally, let $\mathcal{T}$ be the set of AST terms and $\mathcal{N} = \{T\downarrow \mid T \in \mathcal{T}\}$ the set of canonical forms. A *query* is a decidable predicate $Q : \mathcal{N} \to \{\top, \bot\}$.

Q_{arb}: arbitrage detection.. $Q_{\mathsf{arb}}(T\downarrow) = \top$ iff $T\downarrow$ contains a cycle C with $\tau_{\text{in}}(C) =_\tau \tau_{\text{out}}(C)$ and $\delta(C) > 0$. This is the query implemented by our classifier. On tx_R, the canonical form contains one arbitrage cycle with three merged sub-chains, matching entry and exit tokens, and positive delta, so $Q_{\mathsf{arb}}(tx_R\downarrow) = \top$.

Q_{eq}: structural equivalence.. $Q_{\mathsf{eq}}(T_1, T_2) = \top$ iff $T_1\downarrow = T_2\downarrow$. Decidable by Theorem 5; enables clustering transactions by fund flow shape without prior labels. For tx_R and tx_S, $Q_{\mathsf{eq}} = \bot$: their canonical forms share the same star topology but differ in arity (N=3 vs. N=6) and token names. Two transactions with the same arity, the same number of hops per path, and identical token identities would satisfy $Q_{\mathsf{eq}} = \top$. The coarser family query Q_{fam} (below) captures their shared structure.

Q_{match}: strategy matching.. Given a reference transaction T_{ref}, $Q_{\mathsf{match}}(T) = \top$ iff $T{\downarrow} = T_{\mathsf{ref}}{\downarrow}$. With $T_{\mathsf{ref}} = tx_R$, this query flags every transaction whose canonical form exhibits the same three-arm star, regardless of the protocols involved.

Q_{anom}: structural anomaly.. Given a set S of known canonical forms for a bot address, $Q_{\mathsf{anom}}(T) = \top$ iff $T{\downarrow} \notin S$. If the bot behind tx_R later executes a transaction with a four-arm star or a sequential chain, the new canonical form falls outside S and the query flags a strategy change.

Q_{fam}: family classification.. The queries above compare canonical forms by syntactic equality. A coarser equivalence captures the observation that a three-arm star and a five-arm star, or a star with one-hop and two-hop swap paths, are instances of the same *strategy family*. Define the *skeleton map* $\mathsf{sk} : \mathcal{N} \to \mathcal{S}$, where $\mathcal{S}$ is the set of skeleton terms (topology without arity, path length, or token names), inductively:

Let $T{\downarrow} = \mathsf{Star}(\alpha,\, C_1 \parallel \ldots \parallel C_N,\, \alpha)$ be a canonical form with N parallel swap-chain arms, where each $C_i = \mathsf{Swap}(\tau, \tau_{i,1}) \circ \ldots \circ \mathsf{Swap}(\tau_{i,k_i-1}, \tau')$ is a k_i-hop chain. The skeleton $\mathsf{sk}(T{\downarrow}) \in \mathcal{S}$ is computed in three steps:

1) *Erase path length.* Collapse each k_i-hop arm to a single swap retaining only the boundary tokens (analogous to R_1):

$$C_i = \mathsf{Swap}(\tau, \tau_{i,1}) \circ \ldots \circ \mathsf{Swap}(\tau_{i,k_i-1}, \tau') \;\mapsto\; \mathsf{Swap}(\tau, \tau')$$

2) *Erase arity.* Merge the N collapsed arms into a single representative (analogous to R_7):

$$\mathsf{Swap}(\tau, \tau')^{\parallel N} \;\mapsto\; \mathsf{Swap}(\tau, \tau')$$

3) *Erase token identities.* Replace all concrete tokens with a wildcard $\star$:

$$\mathsf{Star}(\alpha,\, \mathsf{Swap}(\tau, \tau'),\, \alpha) \;\mapsto\; \mathsf{Star}(\star,\, \mathsf{Swap}(\star, \star),\, \star)$$

All three steps are decidable (finite tree traversals). Token continuity within each arm ($\tau_{i,k-1}$ feeds into $\tau_{i,k}$) is guaranteed by construction: the chaining rules pair transfers whose junction addresses match, and each swap produces exactly one outgoing token that becomes the next swap's input. Step 1 collapses these junctions, retaining only the boundary tokens. After step 1, all N arms are identical ($\mathsf{Swap}(\tau, \tau')$), so step 2 is a no-op. Step 3 erases the remaining names. The skeleton retains only the topology. Then $Q_{\mathsf{fam}}(T_1, T_2) = \top$ iff $\mathsf{sk}(T_1{\downarrow}) = \mathsf{sk}(T_2{\downarrow})$.

Example: $Q_{\mathsf{fam}}(tx_R, tx_S) = \top$.. We compute sk on both canonical forms.

$tx_R{\downarrow} = \mathsf{Star}(\text{weth},\, \underbrace{C_1 \parallel C_2 \parallel C_3}_{3},\, \text{weth})$ where each C_i is a 2-hop chain:

1) Erase path length (per arm):
   $C_i \mapsto \mathsf{Swap}(\text{weth}, \text{bean}) \quad \forall\, i$
2) Erase arity:
   $\mathsf{Swap}(\text{weth}, \text{bean})^{\parallel 3} \mapsto \mathsf{Swap}(\text{weth}, \text{bean})$
3) Erase tokens:
   $\mathsf{Star}(\text{weth}, \mathsf{Swap}(\text{weth}, \text{bean}), \text{weth})$
   $\mapsto\; \mathsf{Star}(\star,\, \mathsf{Swap}(\star, \star),\, \star)$

$tx_S{\downarrow} = \mathsf{Star}(\text{weth},\, \underbrace{C_1 \parallel \ldots \parallel C_6}_{6},\, \text{weth})$ where each C_i is a 2-hop chain:

1) Erase path length (per arm):
   $C_i \mapsto \mathsf{Swap}(\text{weth}, \text{kabosu}) \quad \forall\, i$
2) Erase arity:
   $\mathsf{Swap}(\text{weth}, \text{kabosu})^{\parallel 6} \mapsto \mathsf{Swap}(\text{weth}, \text{kabosu})$
3) Erase tokens:
   $\mathsf{Star}(\text{weth}, \mathsf{Swap}(\text{weth}, \text{kabosu}), \text{weth})$
   $\mapsto\; \mathsf{Star}(\star,\, \mathsf{Swap}(\star, \star),\, \star)$

Both reduce to $\mathsf{Star}(\star, \mathsf{Swap}(\star, \star), \star)$. Therefore $\mathsf{sk}(tx_R{\downarrow}) = \mathsf{sk}(tx_S{\downarrow})$ and $Q_{\mathsf{fam}}(tx_R, tx_S) = \top$: same family despite different arities (N=3 vs. N=6), different tokens, and different pools.

A hypothetical five-arm arbitrage through longer paths produces the same skeleton. In contrast, a sequential arbitrage (one cycle feeds into the next) would have a different skeleton topology and belong to a different family. The skeleton quotient thus induces a principled taxonomy of MEV strategies directly from the algebraic structure, without manual labeling.

Decidability.. All five queries are decidable: they compose $(\cdot){\downarrow}$ (computable by Theorem 2) with syntactic equality, finite set membership, or the skeleton map (a finite structural traversal), all of which are decidable. Richer queries combining structural predicates with the delta map δ (e.g., "all arbitrages with profit above threshold v through at least 3 pools") remain decidable because δ is computed during normalization and the number of cycles in $T{\downarrow}$ is finite.

The rewriting system thus serves as the semantic foundation for a domain-specific language for fund flow analysis. The present paper instantiates one query (Q_{arb}); the remaining queries, and a systematic study of their expressiveness, are future work.